# UNIVERSITY OF PUNJAB ALLAMA IQBAL CAMPUS

## Roll No : BCSF24M001

Name : Muhammad Huzaifa Farrukh
Section : BCSF24 Morning
Session : 2024-28
Course Name : Computer Networks
Instructor Name : Dr.Madeha Aman

**Date : 03/05/26**

## 1. Project Title

Network Traffic Monitoring and Analysis Platform — **"NetWatch v1.0".**

## 2. Abstract

NetWatch is a real-time network traffic monitoring and analysis platform built using C++ for the backend packet-capture engine and HTML, CSS, and JavaScript for the front-end dashboard. The system uses the Npcap (libpcap-compatible) library to perform live packet sniffing on a selected Wi-Fi or Ethernet interface, parses Ethernet → IPv4 → TCP/UDP/ICMP headers, maps well-known port numbers to their corresponding application services, and streams the captured records to a browser-based dashboard via a lightweight HTTP API. The user can start and stop monitoring on demand, filter records by protocol, source IP, or destination IP, and view live statistical summaries including total packets, per-protocol counts, and average packet size.

## 3. Introduction

Modern computer networks generate enormous volumes of traffic every second. Understanding what is flowing through a network — which protocols dominate, which hosts communicate, and what services are being used — is fundamental to network administration, performance tuning, and cyber-security. This project applies the core concepts of the Computer Networks course (the OSI/TCP-IP layered model, IP addressing, transport-layer protocols, and well-known ports) to build a working tool that captures real packets directly off the wire and presents them in a human-readable form.

Unlike a simulated dataset, NetWatch performs true real-time packet sniffing using the Npcap driver on Windows, which qualifies the project for the bonus criterion mentioned in the project specification.

## 4. Objectives

- Capture live network packets from a chosen network interface.
- Decode Ethernet, IPv4, TCP, UDP, and ICMP headers correctly.
- Map common destination ports to their application-layer services (HTTP, HTTPS, DNS, SSH, etc.).
- Provide Start / Stop monitoring controls through a clean web interface.
- Display per-packet records and live statistics (total, per-protocol, average size).
- Allow filtering of records by protocol, source IP, and destination IP.

## 5. Tools and Technologies Used

| Component | Technology |
|---|---|
| Backend Engine | C++17 with Npcap SDK 1.16 (libpcap-compatible) |
| HTTP Server | Custom lightweight Winsock2 HTTP server (port 8080) |

| | |
|---|---|
| Frontend | HTML5, CSS3, vanilla JavaScript (ES6) |
| Capture Driver | Npcap (Windows packet capture library) |
| Build Tool | g++ (MinGW-w64) with C++17 |
| Platform | Windows 10 / 11 (64-bit) |

**Compilation command:**

```
g++ server.cpp -o server.exe -I"C:/npcap-sdk-1.16/Include" -L"C:/npcap-sdk-1.16/Lib/x64"
lwpcap -lPacket -lws2_32 -ladvapi32 -lpthread -std=c++17 -O2 -D_WIN32_WINNT=0x0601
```

# 6. System Architecture

NetWatch follows a clean two-tier architecture in which the C++ backend performs all heavy network operations and the browser-based frontend handles only presentation and interaction.

## 6.1 Layered Flow

- Layer 1 – Capture: Npcap delivers raw Ethernet frames from the NIC to server.exe.
- Layer 2 – Parsing: server.cpp decodes Ethernet → IPv4 → TCP/UDP/ICMP headers and stores PacketRecord objects.
- Layer 3 – API: An embedded HTTP server exposes /api/devices, /api/start, /api/stop, /api/packets, and /api/stats.
- Layer 4 – Presentation: index.html + app.js poll the API every ~1.5 seconds and update the dashboard.

# 7. Methodology

The project was developed in a structured, phase-by-phase manner following a bottom-up approach — starting from raw packet capture at the lowest layer and building upward toward the user interface.

Phase 1 — Environment Setup: The first step was installing and configuring the Npcap SDK alongside MinGW-w64 on Windows 11. The correct include and library paths were identified, and a minimal test program was written to verify that pcap_findalldevs() could enumerate network interfaces without errors.

Phase 2 — Packet Capture Engine: Once the environment was stable, the packet-capture core was implemented in server.cpp. Packed C structs (EthernetHeader, IPv4Header, TCPHeader, UDPHeader, ICMPHeader) were defined using #pragma pack(push, 1) to map raw byte buffers directly to typed fields with no alignment padding. A pcap_loop callback (packetHandler) was registered to receive each arriving frame. A BPF filter "ip" was compiled and applied so that only IPv4 traffic reached the analyser, reducing processing overhead.

Phase 3 — Protocol Parsing and Service Mapping: Inside packetHandler, the IPv4 protocol field was inspected to branch into TCP, UDP, or ICMP parsing. Port numbers were extracted and looked up in a static std::map<int, std::string> (portToService()) covering 36 well-known services from FTP-Data to MongoDB. Thread-safe global counters and a vector of PacketRecord objects were updated under a std::mutex.

Phase 4 — Embedded HTTP API: A lightweight Winsock2 TCP server was built on port 8080 to bridge the C++ backend and the browser frontend. Each incoming connection is dispatched to a handleClient() thread. The server parses the HTTP method and path, routes to the correct handler (/api/devices, /api/start, /api/stop, /api/packets, /api/stats, /api/clear), and serialises the response as JSON. CORS headers (Access-Control-Allow-Origin: *) were added to permit browser fetch() calls without cross-origin errors.

Phase 5 — Frontend Dashboard: The HTML/CSS/JS frontend was built iteratively. index.html provides the skeleton (header, stats cards, filter bar, packet table, log console). style.css applies a cyber-themed dark palette using CSS custom properties. app.js implements all dynamic behaviour: loadDevices() populates the interface dropdown on page load; startCapture() and stopCapture() call the API; a setInterval polling loop at 1,500 ms fetches fresh packets and statistics; and applyFilter() performs client-side filtering on the cached allPackets array without a new network request.

Phase 6 — Integration and Testing: The backend and frontend were run together and tested with ICMP pings, HTTP browsing, and Wireshark cross-validation. Minor integration issues (CORS preflight handling, double-escaped backslashes in device names, promiscuous-mode rejection on Wi-Fi) were identified and resolved during this phase.

## 8. Implementation Details

### 8.1 Backend (server.cpp)

The backend is implemented in modern C++17. Key elements include:

- Packed structs for EthernetHeader, IPv4Header, TCPHeader, UDPHeader, and ICMPHeader to map raw bytes safely.
- A pcap_loop callback that filters packets using the BPF expression "ip" so only IPv4 traffic enters the analyser.
- A protocol dispatcher that recognises TCP (6), UDP (17), and ICMP (1); all other protocols are tagged as OTHER.
- A port-to-service map that converts numeric ports to readable service names (HTTP, HTTPS, DNS, SSH, FTP, SMTP, etc.).
- Thread-safe shared state protected by std::mutex and std::atomic<bool> for clean Start/Stop control.
- A minimal Winsock2 HTTP server that emits JSON responses consumed by the frontend.

### 8.2 Frontend (index.html, style.css, app.js)

- Clean cyber-themed dashboard with a fixed header, control bar, statistics grid, filter bar, and packets table.
- Device dropdown populated from /api/devices on page load.

- Start, Stop, and Clear buttons that call the corresponding API endpoints.
- Polling loop (setInterval ~1500 ms) that fetches new packets and statistics and re-renders incrementally.
- Client-side filters for Protocol, Source IP, and Destination IP, plus a Reset button.

## 9. Functional Requirements Coverage

| Requirement | Status | Implemented In |
|---|---|---|
| Read / capture network traffic | ✓ | server.cpp (pcap) |
| Display Source IP | ✓ | Packets Table |
| Display Destination IP | ✓ | Packets Table |
| Identify Protocol (TCP/UDP/ICMP) | ✓ | server.cpp parser |
| Map ports to services | ✓ | portToService() |
| Count packets | ✓ | Stats grid |
| Maintain logs | ✓ | Log Console |
| Start / Stop control | ✓ | API + buttons |
| Statistics (total, per-proto, avg size) | ✓ | /api/stats |
| Filter by Protocol / Src IP / Dst IP | ✓ | app.js filters |
| Real-time packet sniffing (BONUS) | ✓ | Npcap live capture |

## 10. Dataset Fields Captured

Each PacketRecord generated by NetWatch contains the following fields, matching the dataset specification in the project brief:

| Field | Example Value |
|---|---|
| Time | 10:35:21 |
| Source IP | 192.168.100.17 |
| Destination IP | 8.8.8.8 |
| Protocol | TCP |
| Source Port | 52341 |

| | |
|---|---|
| Destination Port | 443 |
| Service | HTTPS |
| Packet Size | 512 bytes |

## 11. How to Run

- Install Npcap on Windows (with the “WinPcap-compatible mode” option).
- Compile server.cpp using the g++ command shown in Section 5.
- Run server.exe as Administrator (required for raw packet capture).
- Open index.html in any modern browser.
- Choose the active Wi-Fi interface (Intel® Wi-Fi 6 AX203) from the dropdown and click START.
- Browse any website to observe TCP, UDP, and ICMP packets streaming live into the table.

## 12. Verification of Real-Time Capture

To prove that the capture is genuinely real-time and not a simulation, the following tests are performed during the demonstration:

- ICMP Test: running ping 8.8.8.8 from the command prompt produces exactly four ICMP rows in the dashboard with matching timestamps.
- Airplane-mode Test: toggling Wi-Fi off freezes the packet stream within ~1 second; toggling it back on resumes it instantly.
- HTTP Test: visiting http://neverssl.com produces visible TCP packets on port 80 (service = HTTP).
- Cross-validation with Wireshark: running Wireshark in parallel shows nearly identical packet counts and timing.

## 13. Challenges Faced

Several non-trivial problems were encountered during development, each requiring investigation and a deliberate fix.

- Promiscuous Mode Rejected on Wi-Fi: The initial call to pcap_activate() with promiscuous mode enabled failed on the Intel Wi-Fi 6 AX203 adapter because Windows Wi-Fi drivers typically do not support promiscuous sniffing at the hardware level. This caused pcap_activate() to return a negative error code and abort the capture thread entirely. The fix was to catch this failure, close the handle, and immediately retry with pcap_set_promisc(g_handle, 0) disabled. The adapter still delivers all traffic addressed to the host, so no useful packets are lost in practice.

- Double-Escaped Backslashes in Device Names: Npcap device identifiers on Windows follow the format \\Device\\NPF_{GUID}. When this string was serialised to JSON by the C++ backend and then parsed by JavaScript on the frontend, each double-backslash was interpreted as a single escaped backslash, resulting in a device name with only single backslashes being sent back in the POST /api/start body. The C++ server then received a further-corrupted string and pcap_create() failed silently. The solution was a sanitisation loop at the start of captureThread() that collapses

any run of two consecutive backslashes into one, restoring the correct device path before it is passed to pcap_create().

- CORS Preflight Blocking API Calls: Modern browsers send an HTTP OPTIONS preflight request before any cross-origin POST. Because the backend initially had no OPTIONS handler, the browser received no response and blocked the subsequent POST /api/start call, making the Start button appear broken with no visible error. The fix was to add an explicit OPTIONS route in handleClient() that returns the required Access-Control-Allow-Origin, Access-Control-Allow-Methods, and Access-Control-Allow-Headers headers with an empty 200 body.

- Race Condition on Stop: When the Stop button was clicked, g_capturing was set to false but pcap_next_ex() could still be blocked inside the capture loop waiting for the next timeout. This meant the capture thread could keep running for up to one second after Stop was pressed, and occasionally attempted to write to g_packets after the frontend had already cleared it. The fix was to call pcap_breakloop(g_handle) immediately after setting g_capturing = false, which causes pcap_next_ex() to return -2 and exit the loop cleanly.

## 14. Future Work

- IPv6 Support: The current BPF filter is restricted to "ip" (IPv4 only). Extending it to "ip or ip6" and adding an IPv6Header struct would allow the tool to monitor the growing share of traffic carried over IPv6, including DNS-over-HTTPS and modern streaming protocols.

- Packet Payload Inspection: At present, NetWatch decodes only up to the transport layer. Adding a basic application-layer decoder — for example, extracting the HTTP Host header or the DNS query name from UDP port 53 — would make the Service column far more informative and allow content-based filtering.

- Persistent Logging to Disk: Captured records are kept only in memory and are lost when server.exe is closed or the Clear button is pressed. Writing each PacketRecord to a rotating CSV or SQLite file would enable post-session analysis, long-term traffic trending, and replay of historical captures.

- Graphical Traffic Charts: The statistics grid shows raw counts but provides no visual trend data. Integrating a lightweight charting library (such as Chart.js) into the frontend to display a live rolling time-series of packets-per-second and protocol proportions would make anomaly detection much more intuitive.

- Packet Capture Export (PCAP Format): Adding a /api/export endpoint that returns the captured data as a valid .pcap file would allow users to open their captures directly in Wireshark for deeper analysis without needing to run both tools simultaneously.

- Authentication and Access Control: The HTTP API currently accepts any request from any origin. In a real deployment, adding a simple token-based authentication header and binding the server to 127.0.0.1 only would prevent unauthorised access to potentially sensitive network traffic data.

## 15. Conclusion

NetWatch successfully demonstrates the foundational concepts of network traffic monitoring covered in the Computer Networks course. The project goes beyond the minimum requirements by performing realtime packet sniffing using Npcap, qualifying for the bonus criterion. Through this project, practical understanding was gained of how the OSI/TCP-IP layers cooperate, how IP and transport-layer headers are structured, how libpcap-style capture drivers work, and how a backend service can communicate with a browser front-end through a simple REST-style HTTP API.

## 16. References


- Npcap Project — https://npcap.com
- libpcap Documentation — https://www.tcpdump.org
- Andrew S. Tanenbaum, “Computer Networks”, 5th Edition.
- IANA Service Name and Transport Protocol Port Number Registry.